

AI Sana Challenge Hub

Hackathon MVP built with Next.js App Router, TypeScript, and Tailwind CSS. Step 5 adds publication of confirmed cards into a browser-local catalog with details, sorting, and filters, preserving Steps 1–4.

Run locally

Requires Node.js 20.9 or later and npm.

```
npm ci
npm run dev
```

Open <http://localhost:3000>. Draft creation works without a key. AI clarification requires the server setup below.

```
npm run lint
npm run typecheck
npm run build
npm start
```

`npm start` serves the production build after `npm run build`.

Structure

- `src/app/page.tsx` : homepage and planned flow.
- `src/app/challenges/page.tsx` : published challenge catalog.
- `src/app/challenges/new/page.tsx` : entry point for the draft workspace.
- `src/app/challenges/new/draft-workspace.tsx` : client form, saved-draft list, and editing.
- `src/lib/draft-storage.ts` : browser localStorage reads, validation, and draft updates.
- `src/app/business/page.tsx` : business landing placeholder.

- `src/app/business/proposals/page.tsx` : proposal management placeholder.
- `src/app/students/page.tsx` : student proposal journey placeholder.
- `src/components/` : shared navigation and placeholder presentation.
- `src/types/` : challenge, task card, clarification, readiness, and team proposal contracts.

Keep future behavior close to its route; extract shared logic only when needed. Persistence is browser localStorage only. The only server endpoint is AI clarification. There are no databases, authentication, proposal actions, or team selection.

Step 1: test the draft flow

1. Run `npm run dev`, open <http://localhost:3000>, and click **Create a Challenge**.
2. Click **Save Draft** with an empty or whitespace-only description. A clear validation message appears.
3. Enter a problem description, optionally adding an organization and contact name or email. Save.
4. Verify the saved text, Draft status, creation date, and unique ID. Refresh; the draft remains.
5. Click **Edit draft**, change the description, and save. Refresh; the edit remains and the ID and creation date are unchanged.
6. Return Home and click **Create a Challenge** again to reopen saved drafts. Use **New draft** to create another; select any saved draft from the list.
7. Verify **Improve with AI** is available on the saved draft, after configuring the server key.

Descriptions are saved verbatim, including leading/trailing whitespace and line breaks. Trimming is used only to reject blank descriptions. Optional fields remain absent when empty; no facts are generated. Drafts use UUIDs, ISO creation/update dates, `status: "draft"`, an empty clarification list, and an empty task card.

Storage uses `ai-sana.challenge-drafts.v1`. Save errors retain form text, and unreadable stored data is never silently replaced.

Limitations: drafts belong to the current browser profile and origin (hostname, protocol, and port), do not sync across devices, and disappear when site data is cleared. Save explicitly before refreshing or leaving; there is no autosave. Concurrent editing of the same draft in multiple tabs is not coordinated; the last save wins. There is no deletion/export workflow. Publishing is available for confirmed cards as a fixed local snapshot.

Rules for future implementation

- AI must not invent missing business facts and must ask at least three relevant clarification questions.
- AI-generated content remains editable; manual confirmation is required before publishing.
- Readiness explains its score and missing information. Weights: context/need 20, data/materials 20, expected result 15, success criteria 15, constraints 10, users 10, business communication 10.
- Readiness levels: 0–39 Draft, 40–69 Working, 70–89 Ready, 90–100 Priority.
- Business users manually accept or reject proposals. AI never selects or assigns teams.
- Progress points require a confirmed stage for a selected team.

Types retain the future workflow contracts. Runtime checks cover locally stored drafts and AI clarification responses. No proposals or team selection is implemented.

Step 1 changed files

- Updated `src/app/challenges/new/page.tsx` to render the draft workspace.
- Added `src/app/challenges/new/draft-workspace.tsx` for draft creation and editing.
- Added `src/lib/draft-storage.ts` for local persistence and read validation.
- Updated `src/types/challenge.ts` with optional business name and contact fields.

- Updated `README.md` with current scope, testing instructions, and limitations.

The homepage creation link already pointed to the correct route and required no change. No dependencies were added.

Step 1 validation: lint and TypeScript checks passed. The production build passed with `npm run build -- --webpack`; the default Turbopack build hit this execution environment's port-binding restriction. If that restriction occurs locally, use `npm run dev -- --webpack` or `npm run build -- --webpack`. Existing scripts remain unchanged.

Browser checks passed for empty/whitespace validation, exact multiline text preservation, save → refresh → edit → refresh, unchanged ID/creation date, reopening from Home, and the disabled AI action. Storage checks also covered multiple drafts, failed writes, and malformed-data protection.

Exact Step 0 file inventory

All files below were created; no pre-existing project files were changed.

```
.gitignore
README.md
package.json
package-lock.json
tsconfig.json
next-env.d.ts
next.config.ts
postcss.config.mjs
eslint.config.mjs
src/app/globals.css
src/app/layout.tsx
src/app/page.tsx
src/app/challenges/page.tsx
src/app/challenges/new/page.tsx
src/app/business/page.tsx
src/app/business/proposals/page.tsx
src/app/students/page.tsx
src/components/header.tsx
src/components/placeholder-page.tsx
src/types/challenge.ts
src/types/team-proposal.ts
```

Local generated artifacts (`node_modules`, `.next`, and TypeScript build cache) are ignored. The npm download cache is outside the project in the workspace's `work` directory.

Validation: ESLint passed without warnings, TypeScript passed, production build passed, and all six application routes returned HTTP 200. Browser inspection confirmed homepage content and a 390px layout without horizontal overflow.

Step 2: OpenAI setup

Copy `.env.example` to `.env.local` in this project, then set `OPENAI_API_KEY` there using your own OpenAI API project key. Restart the Next.js server after changing it. Do not paste the key into the draft form or use a `NEXT_PUBLIC_` variable. `.gitignore` already excludes `.env.local` and all `.env*` secrets while allowing the blank `.env.example`.

The server-only `POST /api/clarify` endpoint calls the OpenAI Responses API with `gpt-4o-mini`, strict JSON Schema output, and `store: false`. It sends only the saved problem description, not the contact, organization, or answers. The server validates 3–7 distinct nonempty questions, generates IDs, rejects refusals/incomplete responses, and returns sanitized errors without logging keys or provider payloads. The prompt asks about relevant gaps without inventing business facts. Semantic relevance still requires human review.

Implementation reference: [OpenAI Structured Outputs](#).

Test clarification

1. Save `We want to reduce queues in our university cafeteria.` as a draft.
2. Click **Improve with AI**. With a valid API key and quota, 3–7 relevant questions should appear (typically users, data, expected result, success measures, and constraints).
3. Edit questions or enter answers. They save to localStorage as you type. Refresh and verify they remain.
4. Click **Regenerate questions** after answering. Choose **Keep current questions** to cancel, or explicitly confirm replacement. Existing answers are only replaced after successful generation and persistence.
5. Edit the original description: prior questions/answers stay intact with a notice that they were generated for an earlier description.

- Without a key, click **Improve with AI** and verify the setup message and retry button. The draft remains saved. No fallback questions are supplied.

A request times out after 30 seconds on the server (45 seconds in the browser). Descriptions longer than 12,000 characters remain valid drafts but cannot be sent for clarification. Drafts retain `status: "draft"`; card confirmation is tracked separately and does not publish anything. The draft-only storage format remains backward compatible. The endpoint has no authentication or deployment-grade usage controls, consistent with the local hackathon MVP scope.

Step 2 changed files

- Added `.env.example` (blank server key placeholder).
- Added `src/app/api/clarify/route.ts` (server-only OpenAI endpoint).
- Added `src/lib/clarification.ts` (response and saved-question validation).
- Added `src/app/challenges/new/clarification-panel.tsx` (questions, answers, retry, replacement confirmation).
- Updated `src/app/challenges/new/draft-workspace.tsx` (panel integration and busy-state protection).
- Updated `src/lib/draft-storage.ts` (accept validated saved questions/answers).
- Updated `src/types/challenge.ts` (description snapshot for stale-question notice).
- Added `tests/clarification.test.mjs` (run with `node --test tests/clarification.test.mjs`).
- Updated `README.md` (setup, scope, and validation instructions).

No new dependencies. Existing secret-ignore rules required no changes.

Step 2 verification

Server/storage tests use explicit cafeteria fixtures, not live AI output. They cover strict request schema, minimum/duplicate/empty question validation, provider failures/refusals, old Step 1 drafts, persisted answers after reload, and failed-write preservation. A live OpenAI success check could not be performed because no API key was configured. No generated-question quality or live API success is claimed.

Step 2 checks passed: ESLint, TypeScript, five server/storage tests, and a production build using `npm run build -- --webpack`. Browser checks confirmed the real missing-key/retry flow preserves the cafeteria draft. An isolated provider fixture (visibly labeled `[Test fixture]`, outside the application) verified five rendered questions, exact multiline answers after refresh, regeneration warning/cancellation, simulated API failure preserving the existing questions and answers. A subsequent browser snapshot also showed the answers after refresh; the additional automated field-value check timed out. Step 1 description-edit preservation was covered by the storage tests. Live AI generation remains untested without a key.

Step 3: Business Task Card

No additional AI call is used. Clarification questions are free-form, so keyword guessing could misclassify compound questions. Instead, after answering at least one question, choose a destination field for each answer and click **Create Task Card**. Selected answers are copied verbatim (multiple answers are separated by blank lines). The description fills Need/problem and the optional contact fills Business contact. All other fields stay blank unless explicitly mapped. Unassigned answers remain available above and in source history.

The ten editable fields are Title, Context, Need/problem, Users, Available data/materials, Constraints, Expected result, Success criteria, Business contact, and Communication format. Blank fields show **Needs information** and are allowed at confirmation. Card edits require **Save Card Edits**; clarification answers continue to autosave. Do not refresh with unsaved card edits.

Confirm Card is a separate manual action. Card creation never confirms it. Changing the saved draft, contact, questions/answers, or card content clears confirmation and marks the card **Unconfirmed — needs review**. Existing card text is preserved, so review and adjust it before clicking Confirm Card again. Confirmation persists locally but is not authenticated identity verification. Publishing requires its own deliberate action.

Source revisions are retained separately in `sourceHistory` and viewable in the card's Source history disclosure. This includes earlier descriptions and question/answer values; card editing never rewrites these. Because clarification answers autosave,

history may contain intermediate typing revisions. Browser storage limits still apply; failed writes do not replace saved work.

Step 3 manual test

1. Open a saved draft with at least one clarification answer.
2. Choose an answer destination (for example Users), then **Create Task Card**.
3. Verify all ten fields appear, supplied text is copied exactly, and unsupported fields show Needs information.
4. Edit Title, click **Save Card Edits**, refresh, and verify the title remains.
5. Click **Confirm Card** even with incomplete fields; refresh and verify Confirmed remains.
6. Change a clarification answer. Verify the card immediately becomes Unconfirmed — needs review, keeps its existing text, and allows confirmation again.
7. Review Source history for the prior answer and original draft.
8. Repeat with a draft description or card edit; confirmation must be cleared again.

Step 3 changed files

- Added `src/lib/task-card.ts` : explicit answer mapping, source comparison/snapshots, manual confirmation helper.
- Added `src/app/challenges/new/task-card-panel.tsx` : mapping, all card fields, editing, confirmation, source history.
- Updated `src/app/challenges/new/draft-workspace.tsx` : task-card integration and normalized storage state.
- Updated `src/app/challenges/new/clarification-panel.tsx` : disable source edits while card changes are unsaved; updated next-step text.
- Updated `src/lib/draft-storage.ts` : backward-compatible validation, source history, confirmation invalidation.

- Updated `src/types/challenge.ts` : card creation/review metadata and source history.
- Updated `tests/clarification.test.mjs` : task-card lifecycle and storage regression coverage.
- Updated `README.md` : Step 3 behavior and testing.

No dependencies or API routes changed. No live AI call is needed for card creation.

Step 3 validation passed: ESLint, TypeScript, six server/storage tests, and production build (`npm run build -- --webpack`). Browser testing used previously saved, visibly labeled cafeteria fixtures and verified explicit mapping, all ten fields, incomplete-field labels, card editing and refresh persistence, manual confirmation and refresh persistence, invalidation after changing an answer, review status after refresh, and deliberate reconfirmation. No live AI call was made; card creation is entirely local.

Step 4: deterministic readiness scoring

`calculateReadiness(card)` in `src/lib/readiness.ts` reads only card fields. No AI, draft text outside the card, clarification suggestions, confirmation, or stored score is used. The score is derived on render rather than cached in `localStorage`, so refreshing recalculates it from the saved card without stale scores. Manual card edits update the preview immediately; it shows the signed change relative to the saved card. Save Card Edits persists the inputs; Cancel restores the saved score.

Criterion	Exact point rule
Context and need/problem	Context 10 + Need/problem 10
Available data/materials	20 if supplied
Expected result	15 if supplied
Success criteria	15 if supplied
Constraints	10 if supplied
Users	10 if supplied
Business communication	Business contact 5 + Communication format 5

Each field earns its complete allocation or zero. There are no length, title, or confirmation bonuses. A contact name is sufficient; email syntax and deliverability are not scored. Title always contributes 0 points and can be missing even at 100; the UI still flags it. The seven criteria sum to 100 and the total is clamped to 0–100. This is an information-presence checklist, not an evaluation of truth, detail, feasibility, or quality; any non-placeholder text qualifies. Users should not invent facts to improve it.

For scoring only, text is trimmed, lowercased, and punctuation/whitespace runs become single spaces. Empty or punctuation-only values score zero. The complete normalized placeholder list is: `needs information`, `need information`, `not provided`, `not specified`, `unknown`, `tbd`, `to be determined`, `to be confirmed`, `todo`, `tbc`, `na`, `na`, `none`, `null`, `undefined`, `placeholder`, `enter text here`. Thus `N/A`, `[TBD]`, and `Needs information`. earn zero. Matching is whole-value, not substring; explicit statements such as “No data is available; collect observations” or “No constraints are known” count as supplied information. Original field text is never altered by scoring.

Levels: **0–39 Draft**, **40–69 Working**, **70–89 Ready**, **90–100 Priority**. Current attainable totals are multiples of five; the level function still follows every exact boundary.

The UI includes a live total/level, seven-criterion breakdown, individual field allocations, missing information, and specific suggestions with potential gains. A sticky summary and per-field point labels show the effect while editing. Unsaved manual edits are clearly labeled; no unsaved AI output is scored. Confirmation remains separate: changing a confirmed card marks its UI unconfirmed immediately; saving clears stored confirmation. Canceling unsaved edits restores the unchanged saved card and its prior confirmation.

See a score increase

1. Open an existing task card with Available data/materials blank. Note the score.
2. Enter real information such as “Manual queue observations are available from the cafeteria manager.”
3. The total immediately rises by 20 and shows **+20 points vs saved card**; the data criterion becomes 20/20 and its missing-information suggestion

disappears.

4. Save Card Edits and refresh. The same score is recalculated from the saved fields.
5. Replace that field with `Needs information` to see 20 points removed. Cancel if you do not want to keep the test change.

Step 4 changed files

- Added `src/lib/readiness.ts` : scoring function, weights, presence/placeholder rules, and level boundaries.
- Added `src/app/challenges/new/readiness-panel.tsx` : transparent breakdown and suggestions.
- Updated `src/app/challenges/new/task-card-panel.tsx` : live score, delta, and per-field points.
- Updated `src/app/challenges/new/clarification-panel.tsx` : current feature availability text.
- Updated `src/types/challenge.ts` : scoring contract comments.
- Added `tests/readiness.test.mjs` : seven focused tests including all 512 combinations of weighted-field presence.
- Updated `README.md` : scoring rubric, limitations, and browser instructions.

Run all tests with `node --test tests/*.test.mjs`. No dependencies, API routes, or storage format changes were needed. Proposals and team selection remain out of scope. Publication and catalog are added in Step 5 below.

Step 4 verification: all 13 tests passed (seven readiness tests plus six existing regression tests), ESLint and TypeScript passed, and production build passed with `npm run build -- --webpack`. Browser testing on the existing cafeteria fixture confirmed 20/Draft → 40/Working on filling data/materials, visible +20 unsaved delta, confirmation invalidation, score restoration after save/refresh, placeholder rejection with -20 delta, and Cancel restoring the saved score. No AI call was made.

Step 5: fixed publication and local catalog

After saving and confirming a task card, **Publish Challenge** appears. Any readiness score is allowed, including 0–39 Draft. Unsaved card edits hide this action; saved source/card changes clear confirmation. The storage helper rereads the saved record and rejects stale or unconfirmed publication attempts.

Publication is one localStorage write in the existing `ai-sana.challenge-drafts.v1` collection. It keeps the challenge ID, sets `status: "published"` and an ISO `publishedAt`, and adds `publishedSnapshot` containing only the confirmed card, ID, date, and confirmation metadata. Private draft text and clarification/source history are excluded from the public snapshot. Repeated clicks return the same publication without changing its date or adding an entry. Failed writes leave saved work intact.

The published snapshot is fixed. Later edits affect only the working copy and invalidate its confirmation. Neither those edits nor reconfirmation changes the catalog entry. Step 5 has no update, replacement, or unpublish action. The working copy remains status `published` to indicate that a public snapshot exists; its separate confirmation describes the current working copy. To inspect what students see, use **View published challenge**.

The Challenges page reads snapshots after client hydration, showing Loading first rather than a false empty state. It displays title, need/problem excerpt, shared readiness score/level, and publication date. An empty title displays **Title not provided**, explicitly marking missing content rather than inventing a title. Scores and breakdowns use the existing `calculateReadiness` function on the published card, never the edited working copy.

Default sorting is highest readiness first; lowest first is also available. Equal scores sort by publication date newest first, then ID. Readiness filters include every level. Case-insensitive text search checks the published card fields. The page distinguishes no publications from no filter matches. Each entry opens `/challenges/[id]`, showing all ten published fields and the existing readiness breakdown. Unknown IDs show a browser-local not-found message.

Step 5 browser flow

1. Open a saved card, save edits, click Confirm Card, then Publish Challenge.
2. Open Challenges, open the entry, and refresh its details page. Verify the card, score, date, and fields persist.

3. Publish additional cards at different scores; check descending/ascending order, each readiness filter, text search, and Clear filters.
4. A blank-title, Draft-level card should still publish and appear.
5. Edit a published working copy and save. Confirmation must clear; its original published details and score must remain unchanged.

Step 5 changed files

- Updated `src/types/challenge.ts` : public snapshot type and optional stored snapshot.
- Updated `src/lib/draft-storage.ts` : published-record validation, guarded idempotent publication, immutable snapshot preservation, catalog reads.
- Added `src/lib/catalog.ts` : shared-score sorting and filtering.
- Added `src/app/challenges/use-catalog.ts` : client-only loading and storage/focus refresh.
- Replaced `src/app/challenges/page.tsx` : catalog UI and empty/error states.
- Added `src/app/challenges/[id]/page.tsx` and `challenge-details.tsx` : published details route and browser-loaded card view.
- Updated `src/app/challenges/new/task-card-panel.tsx` : publication action and fixed-snapshot explanation/link.
- Updated `src/app/challenges/new/draft-workspace.tsx` : publish handler and accurate status text.
- Updated `src/app/challenges/new/clarification-panel.tsx` : feature availability text.
- Updated `tests/clarification.test.mjs` : publication and catalog regression tests.
- Updated `README.md` : behavior, checks, and limitations.

Step 5 limitations

"Published" means visible in this browser profile at this origin (protocol/hostname/port), not on a shared online service. Other browsers/devices cannot see these entries; sharing a details URL does not transfer the data. Clearing site data deletes drafts and publications. No authentication, database,

proposals, or team assignment were added. Same-ID duplicate publication is prevented, but simultaneous edits to different records from multiple tabs are not transactionally coordinated. The catalog refreshes on browser focus and storage events. Filters reset to defaults after a page refresh.

Step 5 validation: all 15 tests, lint, TypeScript, and Webpack production build passed. Browser checks on labeled fixtures covered publish → catalog → details → refresh, double-click protection (one entry), three entries at 70/40/10 points, ascending and descending sort, Draft-level filtering, text search, no-match/Clear filters, untitled low-score details, and private edits leaving the published title fixed. HTTP checks confirmed initial catalog/details HTML renders loading rather than an empty-catalog message. Unit tests additionally cover 0-point publication, stale-confirmation rejection, failed-write preservation, and snapshot immutability after reconfirmation. No live AI call was made.